

[Euron] 머신러닝 스터디 1주차

조운서 · 5분 전

통계 수정 삭제



(Euron) 머신러닝

▼ 목록 보기

1/1



교재: 파이썬 머신러닝 완벽 가이드 (권철민)

제 1장. 파이썬 기반의 머신러닝과 생태계 이해

01. 머신러닝의 개념

- 머신러닝: 애플리케이션을 수정하지 않고도 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법

- 현실 세계는 매우 복잡하기 때문에 기존의 소프트웨어 코드만으로 해결하기 어려움
- 머신러닝 알고리즘은 데이터를 기반으로 통계적인 신뢰도를 강화하고 예측 오류를 최소화하기 위한 다양한 수학적 기법을 적용 -> 데이터 내의 숨겨진 패턴을 스스로 인지하고 신뢰도 있는 예측 결과를 도출
- ex) 데이터마이닝, 영상 인식, 음성 인식, 자연어 처리

- 머신러닝 분류

- 지도학습 - 분류, 회귀 등
- 비지도학습 - 클러스터링, 차원 축소 등
- 강화학습

- 머신러닝 단점

- 데이터에 매우 의존적 = 좋은 품질의 데이터를 갖춰야 의미있는 결과를 얻을 수 있음

- 특정 경우, 개발자가 직접 만든 코드보다 정확도 ↓

02. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

- 머신러닝 패키지: 사이킷런
- 행렬/선형대수/통계 패키지: 넘파이, 사이파이
- 데이터 핸들링: 판다스(2차원 데이터 처리)
- 시각화: 맷플롯립, 시본
- 기타: 서드파티 라이브러리, 아이파이썬 등등

03. 넘파이

- 넘파이: 선형대수 기반의 프로그램을 파이썬에서 쉽게 만들 수 있도록 지원하는 대표적인 패키지

- 루프를 사용하지 않고 대량 데이터의 배열 연산 -> 배열 연산 속도 ↑
- 저수준 언어 기반의 호환 API를 제공
- 다양한 데이터 핸들링 기능 제공

넘파이 ndarray 개요

```
import numpy
import numpy as np
```

- 넘파이의 기반 데이터 타입은 **ndarray**
- **array()** : 리스트와 같은 다양한 인자를 입력받아서 ndarray로 변환
- **shape** : ndarray의 크기(= 행과 열의 수)를 튜플 형태로 반환
- **ndim** : ndarray의 차원을 반환

```
array1 = np.array([1, 2, 3])
print('array1 type:', type(array1))
print('array1 array 형태:', array1.shape)
```

```
array2 = np.array([[1, 2, 3],
                  [2, 3, 4]])
print('array2 type:', type(array2))
print('array2 array 형태:', array2.shape)
```

```
array3 = np.array([[1, 2, 3]])
print('array3 type:', type(array3))
print('array3 array 형태:', array3.shape)
```

```
array1 type: <class 'numpy.ndarray'>
array1 array 형태: (3,)
array2 type: <class 'numpy.ndarray'>
array2 array 형태: (2, 3)
array3 type: <class 'numpy.ndarray'>
array3 array 형태: (1, 3)
```

```
print('array1: {0}차원, array2: {1}차원, array3: {2}차원'.format(array1.ndim, array2.ndim, array3.ndim))
array1: 1차원, array2: 2차원, array3: 2차원
```

- array1은 1차원이며 3개의 데이터를 가지고 있음
- array2는 2차원이며 2개의 로우와 3개의 칼럼으로 구성되어 6개의 데이터를 가지고 있음
- array3는 2차원이며 1개의 로우와 3개의 칼럼으로 구성되어 3개의 데이터를 가지고 있음

ndarray의 데이터 타입

- ndarray내의 데이터값은 숫자 값, 문자열 값, 불 값 등이 모두 가능
- ndarray내의 데이터 타입은 같은 데이터 타입만 가능 -> 한 개의 ndarray 객체에 int와 float 공존 불가
- dtype : ndarray내의 데이터 타입 반환
- astype() : 데이터내값의 타입을 인자로 받은 타입으로 변경 -> 메모리 절약 가능

```
list1 = [1, 2, 3]
print(type(list1))
array1 = np.array(list1)
print(type(array1))
print(array1, array1.dtype)
```

```
<class 'list'>
<class 'numpy.ndarray'>
[1 2 3] int64
```

- array1 내의 데이터값은 모두 int64형

```

▶ list2 = [1, 2, 'test']
array2 = np.array(list2)
print(array2, array2.dtype)

list3 = [1, 2, 3.0]
array3 = np.array(list3)
print(array3, array3.dtype)

... ['1' '2' 'test'] <U21
[1. 2. 3.] float64

```

- array2는 int형이 유니코드 문자열 값으로 변환됨
- array3는 int형이 float64형으로 변환됨

```

array_int = np.array([1, 2, 3])
array_float = array_int.astype('float64')
print(array_float, array_float.dtype)

array_int1 = array_float.astype('int32')
print(array_int1, array_int1.dtype)

array_float1 = np.array([1.1, 2.1, 3.1])
array_int2 = array_float1.astype('int32')
print(array_int2, array_int2.dtype)

[1. 2. 3.] float64
[1 2 3] int32
[1 2 3] int32

```

- ndarray 내 데이터값의 타입을 int32로 변경
- float를 int형으로 변경할 때 소수점 이하는 버림

ndarray를 편리하게 생성하기

- **arange()** : 0부터 (함수 인자 값 -1)까지의 값을 순차적으로 ndarray의 데이터값으로 변환
즉, array를 range()로 표현하는 것
- **zeros()** : 함수 인자로 튜플 형태의 shape 값을 입력하면 모든 값을 0으로 채운 ndarray를 반환
- **ones()** : 함수 인자로 튜플 형태의 shape 값을 입력하면 모든 값을 1로 채운 ndarray를 반환

```
sequence_array = np.arange(10)
print(sequence_array)
print(sequence_array.dtype, sequence_array.shape)

sequence_array2 = np.arange(2, 10, 2)
print(sequence_array2)
print(sequence_array2.dtype, sequence_array2.shape)
```

```
[0 1 2 3 4 5 6 7 8 9]
int64 (10,)
[2 4 6 8]
int64 (4,)
```

```
zero_array = np.zeros((3,2), dtype='int32')
print(zero_array)
print(zero_array.dtype, zero_array.shape, '\n\n')
one_array = np.ones((3, 2))
print(one_array)
print(one_array.dtype, one_array.shape)
```

```
[[0 0]
 [0 0]
 [0 0]]
int32 (3, 2)

[[1. 1.]
 [1. 1.]
 [1. 1.]]
float64 (3, 2)
```

ndarray의 차원과 크기 변경

- `reshape()` : 인자로 원하는 크기 값을 입력하면 ndarray를 특정 차원 및 크기로 변환
- `tolist()` : ndarray를 리스트 자료형으로 변환

```

array1 = np.arange(10)
print('array1:\n', array1)

array2 = array1.reshape(2, 5)
print('array2:\n', array2)

array3 = array1.reshape(5, 2)
print('array3:\n', array3)

```

```

array1:
[0 1 2 3 4 5 6 7 8 9]
array2:
[[0 1 2 3 4]
 [5 6 7 8 9]]
array3:
[[0 1]
 [2 3]
 [4 5]
 [6 7]
 [8 9]]

```

- 1차원 array1을 2x5 형태인 2차원 array2로 변환
- 1차원 array1을 5x2 형태인 2차원 array3로 변환

```

array1 = np.arange(10)
print(array1)
array2 = array1.reshape(-1, 5)
print('array2 shape:', array2.shape)
array3 = array1.reshape(5, -1)
print('array3 shape', array3.shape)

```

```

[0 1 2 3 4 5 6 7 8 9]
array2 shape: (2, 5)
array3 shape (5, 2)

```

- 로우 인자에 -1값을 적용한 경우 칼럼에 맞는 로우를 자동으로 새롭게 생성해 변환
- 칼럼 인자에 -1값을 적용한 경우 로우에 맞는 칼럼을 자동으로 새롭게 생성해 변환

넘파이의 ndarray의 데이터 세트 선택하기 - 인덱싱

- 특정한 데이터만 추출: 원하는 위치의 인덱스 값을 지정 -> 해당 위치의 데이터 반환
- 슬라이싱: 연속된 인덱스상의 ndarray를 추출
- 팬시 인덱싱: 일정한 인덱싱 집합을 리스트/ndarray 형태로 지정 -> 해당 위치의 데이터의 ndarray를 반환
- 불린 인덱싱: True/False 값 인덱싱 집합을 기반으로 True에 해당하는 인덱스 위치에 있는 데이터의 ndarray를 반환

- 단일 값 추출

```
# 단일 값 추출
array1 = np.arange(start=1, stop=10)
print('array1:', array1)
value = array1[2]
print('value:', value)
print(type(value))
```

```
array1: [1 2 3 4 5 6 7 8 9]
value: 3
<class 'numpy.int64'>
```

- [2]는 3번째 인덱스 위치의 데이터값을 의미
- array1[2]의 타입은 ndarray가 아닌 ndarray 내의 데이터값

```
array1[0] = 9
array1[8] = 0
print('array1:', array1)
```

```
array1: [9 2 3 4 5 6 7 8 0]
```

- 단일 인덱스를 이용해 데이터값 수정 가능
- 첫 번째 데이터값을 9로 변경
- 9번째 데이터값을 0으로 변경

```
array1d = np.arange(1, 10)
array2d = array1d.reshape(3, 3)
print(array2d, '\n')
```

```
print('(row=0, col=0) index 가리키는 값:', array2d[0, 0])
print('(row=0, col=1) index 가리키는 값:', array2d[0, 1])
print('(row=1, col=0) index 가리키는 값:', array2d[1, 0])
print('(row=2, col=2) index 가리키는 값:', array2d[2, 2])
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
(row=0, col=0) index 가리키는 값: 1
(row=0, col=1) index 가리키는 값: 2
(row=1, col=0) index 가리키는 값: 4
(row=2, col=2) index 가리키는 값: 9
```

- 2차원의 경우 콤마로 분리된 로우와 칼럼 위치의 인덱스를 통해 인덱싱

- 슬라이싱

```
array1 = np.arange(1, 10)
array3 = array1[0:3]
print(array3)
print(type(array3))
```

```
[1 2 3]
<class 'numpy.ndarray'>
```

- [시작 인덱스: 종료 인덱스]는 시작 인덱스에서 (종료 인덱스 -1)의 위치에 있는 데이터의 ndarray를 반환
- 슬라이싱 경우 ndarry 타입

```
array1 = np.arange(1, 10)
array4 = array1[:3]
print(array4)
```

```
array5 = array1[3:]
print(array5)
```

```
array6 = array1[:]
print(array6)
```

```
[1 2 3]
[4 5 6 7 8 9]
[1 2 3 4 5 6 7 8 9]
```

- 슬라이싱 표시할 때 시작, 종료 인덱스 생략 가능
- 시작 인덱스 생략 -> 자동으로 맨 처음 인덱스인 0으로 간주
- 종료 인덱스 생략 -> 자동으로 맨 마지막 인덱스로 간주
- 시작과 종료 인덱스 생략 -> 자동으로 맨 처음/맨 마지막 인덱스로 간주

array2d[0:2, 0:2]	array2d[1:3, 0:3]	array2d[1:3, :]																											
<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9
1	2	3																											
4	5	6																											
7	8	9																											
1	2	3																											
4	5	6																											
7	8	9																											
1	2	3																											
4	5	6																											
7	8	9																											
array2d[:, :]	array2d[:2, 1:]	array2d[:2, 0]																											
<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9	<table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9
1	2	3																											
4	5	6																											
7	8	9																											
1	2	3																											
4	5	6																											
7	8	9																											
1	2	3																											
4	5	6																											
7	8	9																											

- 2차원의 경우 콤마로 로우와 칼럼 인덱스를 지칭

```
print(array2d[0])
print(array2d[1])
print('array2d[0] shape:', array2d[0].shape,
      '\narray2d[1] shape:', array2d[1].shape)
```

```
[1 2 3]
[4 5 6]
array2d[0] shape: (3,)
array2d[1] shape: (3,)
```

- 2차원 ndarray에서 뒤에 오는 인덱스를 없애면 1차원 ndarray를 반환

- `array2d[0]`의 경우 `array2d`의 첫 번째 로우 ndarray를 반환

- 팬시 인덱싱

array2d[0,1, 2]		
1	2	3
4	5	6
7	8	9

array2d[0, 1, 0:2]		
1	2	3
4	5	6
7	8	9

array2d[0, 1]		
1	2	3
4	5	6
7	8	9

- 불린 인덱싱

```
array1d = np.arange(1, 10)
```

```
array3 = array1d[array1d>5]
```

```
print('array1d > 5 불린 인덱싱 결과 값:', array3)
```

```
array1d > 5 불린 인덱싱 결과 값: [6 7 8 9]
```

```
array1d > 5
```

```
array([False, False, False, False, False,  True,  True,  True,  True])
```

- []내에 조건문을 그대로 기재 -> 필터링
- True 값이 있는 위치 인덱스 값으로 자동 변환해 해당하는 인덱스 위치의 데이터를 반환

행렬의 정렬

- `np.sort()`: 원 행렬을 유지한 채 원 행렬의 정렬된 행렬을 반환
- `ndarray.sort()`: 원 행렬 자체를 정렬한 형태로 변환 + None을 반환
- `np.argsort()`: 정렬 행렬의 원본 행렬 인덱스를 ndarray 형으로 반환

```
org_array = np.array([3, 1, 9, 5])
print('원본 행렬:', org_array)

sort_array1 = np.sort(org_array)
print('np.sort() 호출 후 반환된 정렬 행렬:', sort_array1)
print('np.sort() 호출 후 원본 행렬:', org_array)

sort_array2 = org_array.sort()
print('org_array.sort() 호출 후 반환된 행렬:', sort_array2)
print('org_array.sort() 호출 후 원본 행렬:', org_array)
```

```
원본 행렬: [3 1 9 5]
np.sort() 호출 후 반환된 정렬 행렬: [1 3 5 9]
np.sort() 호출 후 원본 행렬: [3 1 9 5]
org_array.sort() 호출 후 반환된 행렬: None
org_array.sort() 호출 후 원본 행렬: [1 3 5 9]
```

- np.sort(org_array)는 org_array 행렬을 변경하지 않고 정렬된 형태로 반환
- org_array.sort()는 원본 행렬 자체를 정렬한 값으로 변환

```
sort_array1_desc = np.sort(org_array)[::-1]
print('내림차순으로 정렬:', sort_array1_desc)
```

```
내림차순으로 정렬: [9 5 3 1]
```

- [::-1] 을 사용하면 내림차순으로 정렬 가능

```
array2d = np.array([[8, 12],
                    [7, 1]])

sort_array2d_axis0 = np.sort(array2d, axis=0)
print('로우 방향으로 정렬:\n', sort_array2d_axis0)

sort_array2d_axis1 = np.sort(array2d, axis=1)
print('칼럼 방향으로 정렬:\n', sort_array2d_axis1)
```

```
로우 방향으로 정렬:
[[ 7  1]
 [ 8 12]]
칼럼 방향으로 정렬:
[[ 8 12]
 [ 1  7]]
```

- 2차원 이상일 경우 axis 축 값 설정을 통해 방향을 선택해 정렬
- axis=0 은 로우, axis=1 은 칼럼 방향

```
import numpy as np

name_array = np.array(['John', 'Mike', 'Sarah', 'Kate', 'Samuel'])
score_array = np.array([78, 95, 84, 98, 88])

sort_indices_asc = np.argsort(score_array)
print('성적 오름차순 정렬 시 score_array의 인덱스:', sort_indices_asc)
print('성적 오름차순으로 name_array의 이름 출력:', name_array[sort_indices_asc])
```

```
성적 오름차순 정렬 시 score_array의 인덱스: [0 2 4 1 3]
성적 오름차순으로 name_array의 이름 출력: ['John' 'Sarah' 'Samuel' 'Mike' 'Kate']
```

- `np.argsort()` 을 이용해 반환된 인덱스를 다른 ndarray에 팬시 인덱스로 적용해 추출

선형대수 연산 - 행렬 내적과 전치 행렬 구하기

- `np.dot(A, B)` : A행렬과 B행렬의 내적(= 행렬 곱)을 반환
- `np.transpose(A)` : 행렬 A의 전치 행렬을 반환

04. 판다스

- 판다스: 파이썬의 대표적인 데이터 핸들링 프레임워크

- Series: 칼럼이 하나 뿐인 데이터 구조체 + Index를 key 값으로 가짐
- DataFrame: 칼럼이 여러 개인 데이터 구조체 즉 + Index를 key 값으로 가짐
- 모든 DataFrame 내의 데이터는 생성되는 순간 고유의 Index 값을 가짐

판다스 시작

```
import pandas
import pandas as pd
```

- `pd.read_csv('파일명')` : 인자로 들어온 파일을 로딩해 DataFrame 객체로 반환
↳ 별다른 파라미터 지정이 없으면 파일의 맨 처음 로우를 칼럼명으로 인지하고 칼럼으로 변환
- `head(N)` : DataFrame의 맨 앞에 있는 N개의 로우 반환(디폴트: 5개)
- `info()` : 총 데이터 건수와 데이터 타입, Null 건수를 출력
- `describe()` : 칼럼별 숫자형 데이터값의 n-percentile 분포도, 평균값, 최댓값, 최솟값을 출력
↳ 오직 숫자형 칼럼의 분포도만 조사(object 타입의 칼럼은 자동으로 출력에서 제외)
- `value_counts()` : 해당 칼럼값의 유형과 건수를 출력

	A	B	C	D	E	F	G	H	I	J	K	L
1	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	0	3	Braund, M	male	22	1	0	A/5 21171	7.25		S
3	2	1	1	Cummings, J	female	38	1	0	PC 17599	71.2833	C85	C
4	3	1	3	Heikkinen, female		26	0	0	STON/O2.	7.925		S
5	4	1	1	Futrelle, M	female	35	1	0	113803	53.1	C123	S
6	5	0	3	Allen, Mr.	male	35	0	0	373450	8.05		S
7	6	0	3	Moran, Mr	male		0	0	330877	8.4583		Q
8	7	0	1	McCarthy, male		54	0	0	17463	51.8625	E46	S
9	8	0	3	Palsson, M	male	2	3	1	349909	21.075		S
10	9	1	3	Johnson, M	female	27	0	2	347742	11.1333		S
11	10	1	2	Nasser, M	female	14	1	0	237736	30.0708		C
12	11	1	3	Sandstrom	female	4	1	1	PP 9549	16.7	G6	S
13	12	1	1	Bonnell, M	female	58	0	0	113783	26.55	C103	S
14	13	0	3	Saunders	male	20	0	0	A/5. 2151	8.05		S
15	14	0	3	Andersson	male	39	1	5	347082	31.275		S
16	15	0	3	Vestrom, M	female	14	0	0	350406	7.8542		S
17	16	1	2	Hewlett, M	female	55	0	0	248706	16		S
18	17	0	3	Rice, M	male	2	4	1	302652	29.125		Q

```
titanic_df = pd.read_csv('titanic_train.csv')
print('titanic 변수 type:', type(titanic_df))
titanic_df
```

titanic 변수 type: <class 'pandas.core.frame.DataFrame'>

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May)	female	35.0	1	0	113803	53.1000

- 데이터 파일의 첫 번째 줄에 있던 칼럼 문자열이 DataFrame의 칼럼으로 해당됨

```
titanic_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

- 전체 데이터는 891개의 row, 12개의 column을 가짐
- 마지막에는 전체 12개의 칼럼들의 타입을 요약하여 출력

```
titanic_df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

- 숫자형 칼럼의 분포도만 출력

```
value_counts = titanic_df['Pclass'].value_counts()
print(value_counts)
```

```
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

```
titanic_pclass = titanic_df['Pclass']
print(type(titanic_pclass))
```

```
<class 'pandas.core.series.Series'>
```

- Pclass값은 3이 491개, 1이 216개, 2가 184개로 구성되어 있음
- DataFrame의 [] 연산자 내부에 칼럼명을 입력하면 Series 객체를 반환

```
import numpy as np
```

```
col_name1 = ['col1']
list1 = [1, 2, 3]
array1 = np.array(list1)
print('array1 shape:', array1.shape)
```

```
df_list1 = pd.DataFrame(list1, columns=col_name1)
print('1차원 리스트로 만든 DataFrame:\n', df_list1)
```

```
df_array1 = pd.DataFrame(array1, columns=col_name1)
print('1차원 ndarray로 만든 DataFrame:\n', df_array1)
```

```
array1 shape: (3,)
1차원 리스트로 만든 DataFrame:
   col1
0      1
1      2
2      3
1차원 ndarray로 만든 DataFrame:
   col1
0      1
1      2
2      3
```

- value_counts() 는 Null 값을 무시하고 출력 -> dropna 인자값을 False로 입력하면 Null 값 포함하여 계산

DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환

- 판다스 DataFrame 객체의 생성 인자 data는 리스트나 딕셔너리 또는 넘파이 ndarray를 입력받고, 생성 인자 columns는 칼럼명 리스트를 입력받아 DataFrame을 생성할 수 있음
- 칼럼명을 지정하지 않으면 자동으로 칼럼명 할당
- 2차원 이하의 데이터들만 DataFrame으로 변환 가능
- `to_dict()` : DataFrame을 딕셔너리로 변환

```
col_name2=['col1', 'col2', 'col3']

list2 = [[1, 2, 3],
         [11, 12, 13]]
array2 = np.array(list2)
print('array2 shape:', array2.shape)

df_list2 = pd.DataFrame(list2, columns=col_name2)
print('2차원 리스트로 만든 DataFrame:\n', df_list2)

df_array2 = pd.DataFrame(array2, columns=col_name2)
print('2차원 ndarray로 만든 DataFrame:\n', df_array2)
```

```
array2 shape: (2, 3)
2차원 리스트로 만든 DataFrame:
   col1  col2  col3
0     1     2     3
1    11    12    13
2차원 ndarray로 만든 DataFrame:
   col1  col2  col3
0     1     2     3
1    11    12    13
```

- 1차원 형태의 데이터를 기반으로 DataFrame을 생성 -> 칼럼 1개만 필요

```
dict = {'col1':[1, 11], 'col2':[2, 22], 'col3':[3, 33]}
df_dict = pd.DataFrame(dict)
print('딕셔너리로 만든 DataFrame:\n', df_dict)
```

```
딕셔너리로 만든 DataFrame:
   col1  col2  col3
0     1     2     3
1    11    22    33
```

- 2차원 형태의 데이터를 기반으로 DataFrame을 생성
- 2행 3열 형태 -> 칼럼 3개 필요

```

array3 = df_dict.values
print('df_dict.values 타입:', type(array3),
      'df_dict.values shape:', array3.shape)
print(array3)

df_dict.values 타입: <class 'numpy.ndarray'> df_dict.values shape: (2, 3)
[[ 1  2  3]
 [11 22 33]]

list3 = df_dict.values.tolist()
print('df_dict.values.tolist() 타입:', type(list3))
print(list3)

dict3 = df_dict.to_dict('list')
print('\n df_dict.to_dict() 타입:', type(dict3))
print(dict3)

df_dict.values.tolist() 타입: <class 'list'>
[[1, 2, 3], [11, 22, 33]]

df_dict.to_dict() 타입: <class 'dict'>
{'col1': [1, 11], 'col2': [2, 22], 'col3': [3, 33]}

```

```

titanic_df['Age_0'] = 0
titanic_df.head(3)

```

id	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_0
0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	0
1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C	0
1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	0

DataFrame의 칼럼 데이터 세트 생성과 수정

- [] 연산자를 이용해 쉽게 데이터 세트 생성과 수정 가능

Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_0	Age_by_10	Family_No
male	22.0	1	0	A/5 21171	7.2500	NaN	S	0	220.0	2
female	38.0	1	0	PC 17599	71.2833	C85	C	0	380.0	2
female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	0	260.0	1

을 단계: [titanic_df 변수로 코드 생성](#) [New interactive sheet](#)

```
titanic_df['Age_by_10'] = titanic_df['Age_by_10'] + 100
titanic_df.head(3)
```

Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_0	Age_by_10	Family_No
male	22.0	1	0	A/5 21171	7.2500	NaN	S	0	320.0	2
female	38.0	1	0	PC 17599	71.2833	C85	C	0	480.0	2
female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	0	360.0	1

- 새로운 칼럼명 'Age_0'으로 모든 데이터값이 0으로 할당된 Series가 기존 DataFrame에 추가됨

```
titanic_df['Age_by_10'] = titanic_df['Age']*10
titanic_df['Family_No'] = titanic_df['SibSp'] + titanic_df['Parch']+1
titanic_df.head(3)
```

Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_0	Age_by_10	Family_No
male	22.0	1	0	A/5 21171	7.2500	NaN	S	0	220.0	2
female	38.0	1	0	PC 17599	71.2833	C85	C	0	380.0	2
female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	0	260.0	1

- 'Age_by_10' 칼럼 값을 일괄적으로 기존 값+100으로 업데이트

DataFrame 데이터 삭제

- `drop()` : 데이터를 삭제

```
DataFrame.drop(labels=None, axis=0, index=None, columns=None, level=None, inplace=False)
```



```
titanic_drop_df = titanic_df.drop('Age_0', axis=1)
titanic_drop_df.head(3)
```

Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_by_10	Family_No
Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S	320.0	2
Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.2833	C85	C	480.0	2
Heikinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	360.0	1

```
titanic_df.head(3)
```

Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	Age_0	Age_by_10	Family_No
male	22.0	1	0	A/5 21171	7.2500	NaN	S	0	320.0	2
female	38.0	1	0	PC 17599	71.2833	C85	C	0	480.0	2
female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S	0	360.0	1

```
drop_result = titanic_df.drop(['Age_0', 'Age_by_10', 'Family_No'], axis=1, inplace=True)
print('inplace=True 로 drop 후 반환된 값:', drop_result)
titanic_df.head(3)
```

inplace=True 로 drop 후 반환된 값: None

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S

- axis=1 을 이용하여 'Age_0' 칼럼을 삭제
- 원본 데이터에 적용X -> inplace=True 를 사용하면 원본이 바뀜
- inplace=True 로 설정하면 None을 반환

```
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', 15)
print('### before axis 0 drop ###')
print(titanic_df.head(3))

titanic_df.drop([0, 1, 2], axis=0, inplace=True)

print('### after axis 0 drop ###')
print(titanic_df.head(3))
```

```
### before axis 0 drop ###
PassengerId  Survived  Pclass      Name  Sex  Age  SibSp  Parch    Ticket   Fare Cabin Embarked
0            1         0       3  Braund, Mr....  male  22.0     1     0      A/5 21171   7.2500   NaN      S
1            2         1       1  Cumings, Mr... female  38.0     1     0      PC 17599  71.2833   C85      C
2            3         1       3  Heikkinen, ... female  26.0     0     0  STON/O2. 31...   7.9250   NaN      S

### after axis 0 drop ###
PassengerId  Survived  Pclass      Name  Sex  Age  SibSp  Parch    Ticket   Fare Cabin Embarked
3            4         1       1  Futrelle, M... female  35.0     1     0  113803   53.1000  C123      S
4            5         0       3  Allen, Mr. ... male   35.0     0     0  373450   8.0500   NaN      S
5            6         0       3  Moran, Mr. ... male   NaN     0     0  330877   8.4583   NaN      Q
```

- axis=0 을 이용해 Index 0, 1, 2에 위치한 로우를 삭제

Index 객체

- **Index**: DataFrame, Series의 레코드를 고유하게 식별하는 객체

↳ Index 객체의 값을 함부로 변경할 수 **없음**

↳ Index는 연산에서 제외/**오직 식별용**

- DataFrame.index : DataFrame에서 Index 객체만 추출
- Series.index : Series에서 Index 객체만 추출
- **reset_index()** : 기존 인덱스를 열로 바꾸고 새 인덱스를 설정

```
titanic_df = pd.read_csv('titanic_train.csv')

indexes = titanic_df.index
print(indexes)
print('Index 객체 arrat 값:\n', indexes.values)
```

```
RangeIndex(start=0, stop=891, step=1)
Index 객체 arrat 값:
: [ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17
   18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
   36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
   54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71
   72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89
   90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107
  108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
  126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
  144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161
  162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179
  180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
  198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215
  216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233
  234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251
  252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269
  270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287
  288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305
  306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323
  324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341
  342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359
  360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377
  378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395
  396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413
  414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431
  432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
  451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469
  470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487
  488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505
  506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523
  524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541
  542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559
  560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577
  578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595
  596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613
  614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631
  632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649
  650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667
  668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685
  686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703
  704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721
  722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739
  740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757
  758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775
  776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793
  794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811
  812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829
  830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847
  848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865
  866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883
  884 885 886 887 888 889 890 891]
```

```
print(type(indexes.values))
print(indexes.values.shape)
print(indexes[:5].values)
print(indexes.values[:5])
print(indexes[6])
```

```
<class 'numpy.ndarray'>
(891,)
[0 1 2 3 4]
[0 1 2 3 4]
6
```

- Index 객체는 식별성 데이터를 1차원 array로 가짐
- 단일 값 반환 및 슬라이싱 가능

```
titanic_reset_df = titanic_df.reset_index(inplace=False)
titanic_reset_df.head(3)
```

	index	PassengerId	Survived	Pclass	Name	Sex
0	0	1	0	3	Braund, Mr....	male
1	1	2	1	1	Cumings, Mr...	female
2	2	3	1	3	Heikkinen, ...	female

- 새롭게 인덱스를 연속 숫자 형으로 할당
- 기존 index는 index라는 칼럼으로 추가됨

데이터 선택 및 필터링

- DataFrame 바로 뒤의 [] 연산자는 넘파이의 [] 나 Series의 [] 와 다름

넘파이 [] 연산자	DataFrame [] 연산자
행/열의 위치, 슬라이싱 범위 등 가능	칼럼명 문자, 인덱스로 변환 가능한 표현식

- DataFrame 바로 뒤의 [] 내 입력값은 칼럼명(또는 칼럼의 리스트)을 지정해 칼럼 지정 연산에 사용하거나 불린 인덱스 용도로만 사용

-> DataFrame 뒤에 있는 [] 는 칼럼만 지정할 수 있는 '칼럼 지정 연산자'

```
titanic_df[0:2]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr....	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mr...	female	38.0	1	0	PC 17599	71.2833	C85	C

```
titanic_df[titanic_df['Pclass']==3].head(3)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr....	male	22.0	1	0	A/5 21171	7.250	NaN	S
2	3	1	3	Heikkinen, ...	female	26.0	0	0	STON/O2. 31...	7.925	NaN	S
4	5	0	3	Allen, Mr. ...	male	35.0	0	0	373450	8.050	NaN	S

- 인덱스 형태로 변환 가능한 표현식은 [] 내에 입력 가능하지만 쓰지 않는 게 좋음

- DataFrame iloc[] 연산자

- `iloc[행 위치 정숫값, 열 위치 정숫값]` : 해당 **위치**에 있는 데이터를 반환

iloc [] 연산 유형	설명
<code>df.iloc[1, 0]</code>	두번째 행의 첫번째 열 위치에 있는 단일 값 반환
<code>df.iloc[2, 1]</code>	세번째 행의 두번째 열 위치에 있는 단일 값 반환
<code>df.iloc[0:2, 0:1]</code>	첫번째~두번째 행과 첫번째, 두번째 열에 해당하는 DataFrame 반환
<code>df.iloc[0:2, 0:3]</code>	첫번째~두번째 행의 첫번째~세번째 열 범위에 해당하는 DataFrame 반환
<code>df.iloc[:]</code>	전체 DataFrame 반환
<code>df.iloc[:, :]</code>	전체 DataFrame 반환

- 불린 인덱싱 제공X

- DataFrame loc [] 연산자

- `loc[인덱스값, 칼럼명]` : **명칭** 기반으로 해당 데이터 추출

loc [] 연산 유형	설명
<code>df.loc['three', 'Name']</code>	인덱스 값 three인 행의 Name 칼럼의 단일값 반환
<code>df.iloc['one':'two', ['Name', 'Year']]</code>	인덱스 값 one~two 행의 Name과 Year 칼럼에 해당하는 DataFrame 반환
<code>df.iloc['one':'three', 'Name':'Gender']</code>	인덱스 값 one~three 행의 Name~Gender 칼럼까지의 DataFrame 반환
<code>df.loc[:]</code>	전체 DataFrame 반환

- 불린 인덱싱 가능

- 불린 인덱싱

```
titanic_df[titanic_df['Age']>60][['Name', 'Age']].head(3)
```

	Name	Age
33	Wheadon, Mr...	66.0
54	Ostby, Mr. ...	65.0
96	Goldschmidt...	71.0

```
titanic_df.loc[titanic_df['Age']>60, ['Name', 'Age']].head(3)
```

	Name	Age
33	Wheadon, Mr...	66.0
54	Ostby, Mr. ...	65.0
96	Goldschmidt...	71.0

- [] 나 loc[] 에서 사용 가능

```
titanic_df[(titanic_df['Age']>60) & (titanic_df['Pclass']==1) &
           (titanic_df['Sex']=='female')]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
275	276	1	1	Andrews, Mi...	female	63.0	1	0	13502	77.9583
829	830	1	1	Stone, Mrs....	female	62.0	0	0	113572	80.0000

```
cond1 = titanic_df['Age'] > 60
cond2 = titanic_df['Pclass'] == 1
cond3 = titanic_df['Sex'] == 'female'
titanic_df[cond1 & cond2 & cond3]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fa
275	276	1	1	Andrews, Mi...	female	63.0	1	0	13502	77.95
829	830	1	1	Stone, Mrs....	female	62.0	0	0	113572	80.00

- | 복합 조건 연산자 | 의미 |
|-----------|-----|
| & | and |
| | or |
| ~ | Not |

- 개별 조건은 () 로 묶음

정렬

- sort_values()

- `sort_values()` : DataFrame과 Series를 정렬
- by 로 특정 칼럼을 입력하면 해당 칼럼으로 정렬
- ascending=False 로 설정하면 내림차순으로 정렬(기본은 ascending=True)
- inplace=True 로 설정하면 원본에 정렬 결과 그대로 적용(기본은 inplace=False)

```
titanic_sorted = titanic_df.sort_values(by=['Pclass', 'Name'], ascending=False)
titanic_sorted.head(3)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin
868	869	0	3	van Melkebe...	male	NaN	0	0	345777	9.5	NaN
153	154	0	3	van Billiar...	male	40.5	0	2	A/5. 851	14.5	NaN
282	283	0	3	de Pelsmaek...	male	16.0	0	0	345778	9.5	NaN

- 'Pclass'와 'Name' 칼럼으로 내림차순 정렬

- Aggregation 함수 적용

- DataFrame에서 바로 aggregation을 호출할 경우 모든 칼럼에 해당 aggregation을 적용

```
titanic_df.count()
```

```
0
PassengerId  891
Survived     891
Pclass       891
Name         891
Sex          891
Age         714
SibSp        891
Parch        891
Ticket       891
Fare         891
Cabin        204
Embarked     889
```

```
dtype: int64
```

- 모든 칼럼에 `count()` 가 적용됨

```
titanic_df[['Age', 'Fare']].mean()
```

```
0
Age    29.699118
Fare    32.204208
```

```
dtype: float64
```

- 'Age'와 'Fare' 칼럼만 불러와 평균을 구함

- `groupby()` 적용

- `groupby()` : 특정 칼럼 기준으로 그룹화
- `agg()` : 그룹별로 여러 집계 연산을 수행

```
titanic_groupby = titanic_df.groupby(by='Pclass')
print(type(titanic_groupby))
```

```
<class 'pandas.core.groupby.generic.DataFrameGroupBy'>
```

- Pclass 칼럼 기준으로 GroupBy된 DataFrameGroupby 객체 반환

```
titanic_groupby = titanic_df.groupby('Pclass').count()
titanic_groupby
```

	PassengerId	Survived	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
Pclass											
1	216	216	216	216	186	216	216	216	216	176	214
2	184	184	184	184	173	184	184	184	184	16	184
3	491	491	491	491	355	491	491	491	491	12	491

- Pclass 칼럼을 제외한 모든 칼럼에 `count()` 적용

```
titanic_df.groupby('Pclass')['Age'].agg([max, min])
```

/tmp/ipykernel_1872/1565262784.py:1: FutureWarning: Th
titanic_df.groupby('Pclass')['Age'].agg([max, min])
/tmp/ipykernel_1872/1565262784.py:1: FutureWarning: Th
titanic_df.groupby('Pclass')['Age'].agg([max, min])

	max	min
Pclass		
1	80.0	0.92
2	70.0	0.67
3	74.0	0.42

- agg() 를 적용해 Pclass 기준으로 Age의 최댓값과 최솟값을 담은 DataFrameGroupby 객체 반환

```
agg_format = {'Age': 'max', 'SibSp': 'sum', 'Fare': 'mean'}
titanic_df.groupby('Pclass').agg(agg_format)
```

	Age	SibSp	Fare
Pclass			
1	80.0	90	84.154687
2	70.0	74	20.662183
3	74.0	302	13.675550

- agg() 내에 입력값으로 딕셔너리 형태로 aggregation이 적용될 칼럼들과 aggregation 함수 입력

결손 데이터 처리하기

- 결손 데이터: 칼럼에 값이 없는, 즉 NULL인 경우

- 넘파이의 NaN으로 표시
- NaN 값은 평균, 총합 등의 함수 연산 시 제외됨
- `isna` : 데이터가 NaN인지 아닌지 알려줌
- `fillna()` : 결손 데이터를 다른 값으로 대체

```
titanic_df.isna().head(3)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0		False	False	False	False	False	False	False	False	False	True	False
1		False	False	False	False	False	False	False	False	False	False	False
2		False	False	False	False	False	False	False	False	False	True	False

```
titanic_df.isna().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

- Age에 177개, Cabin에 687개, Embarked에 2개의 결손 데이터가 있음

```
titanic_df['Age'] = titanic_df['Age'].fillna(titanic_df['Age'].mean())
titanic_df['Embarked'] = titanic_df['Embarked'].fillna('S')
titanic_df.isna().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0

dtype: int64

- Age 칼럼의 NaN 값을 평균 나이로, Embarked 칼럼의 NaN 값을 'S'로 대체

apply lambda 식으로 데이터 가공

- apply lambda: `apply()` 로 각 값에 `lambda` 함수 적용


```
titanic_df['Child_Adult'] = titanic_df['Age'].apply(lambda x: 'Child' if x <= 15 else 'Adult')
titanic_df[['Age', 'Child_Adult']].head(8)
```

	Age	Child_Adult
0	22.000000	Adult
1	38.000000	Adult
2	26.000000	Adult
3	35.000000	Adult
4	35.000000	Adult
5	29.699118	Adult
6	54.000000	Adult
7	2.000000	Child

- 나이가 15세 미만이면 'Child', 그렇지 않으면 'Adult'로 구분하는 새로운 칼럼 'Child_Adult' 생성

```
titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x: 'Child' if x<=15
                                                else ('Adult' if x <= 60 else 'Elderly')
titanic_df['Age_cat'].value_counts()
```

	count
Age_cat	
Adult	786
Child	83
Elderly	22

dtype: int64

```
def get_category(age):
    cat = ''
    if age <= 5: cat = 'Baby'
    elif age <= 12: cat = 'Child'
    elif age <= 18: cat = 'Teenager'
    elif age <= 25: cat = 'Student'
    elif age <= 35: cat = 'Young Adult'
    elif age <= 60: cat = 'Adult'
    else: cat = 'Elderly'

    return cat

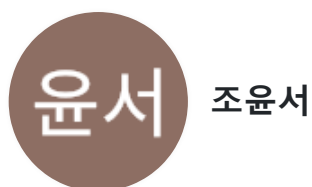
titanic_df['Age_cat'] = titanic_df['Age'].apply(lambda x: get_category(x))
titanic_df[['Age', 'Age_cat']].head()
```

	Age	Age_cat
0	22.0	Student
1	38.0	Adult
2	26.0	Young Adult
3	35.0	Young Adult
4	35.0	Young Adult

- else if는 지원 X
- else if를 이용하기 위해 else 절을 ()로 내포해 () 내에서 다시 if else 적용해 사용
- else if가 많이 나와야하는 경우, 별도의 함수를 만드는 게 나음

05. 정리

- **머신러닝**: 애플리케이션을 수정하지 않고도 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법
- **넘파이**: 선형대수 기반의 프로그램을 파이썬에서 쉽게 만들 수 있도록 지원하는 대표적인 패키지
 - `array()`: 리스트와 같은 다양한 인자를 입력받아서 ndarray로 변환
 - `shape`: ndarray의 크기(= 행과 열의 수)를 튜플 형태로 반환
 - `arange()`: 0부터 (함수 인자 값 -1)까지의 값을 순차적으로 ndarray의 데이터값으로 변환
 - `reshape()`: 인자로 원하는 크기 값을 입력하면 ndarray를 특정 차원 및 크기로 변환
 - `tolist()`: ndarray를 리스트 자료형으로 변환
- **판다스**: 파이썬의 대표적인 데이터 핸들링 프레임워크
 - `pd.read_csv('파일명')`: 인자로 들어온 파일을 로딩해 DataFrame 객체로 반환
 - `head(N)`: DataFrame의 맨 앞에 있는 N개의 로우 반환(디폴트: 5개)
 - `info()`: 총 데이터 건수와 데이터 타입, Null 건수를 출력
 - `describe()`: 칼럼별 숫자형 데이터값의 n-percentile 분포도, 평균값, 최댓값, 최솟값을 출력
 - `value_counts()`: 해당 칼럼값의 유형과 건수를 출력
 - `drop()`: 데이터를 삭제
 - `iloc[행 위치 정숫값, 열 위치 정숫값]`: 해당 **위치**에 있는 데이터를 반환
 - `loc[인덱스값, 칼럼명]`: **명칭** 기반으로 해당 데이터 추출
 - `sort_values()`: DataFram과 Series를 정렬
 - `by` 로 특정 칼럼을 입력하면 해당 칼럼으로 정렬
 - `groupby()`: 특정 칼럼 기준으로 그룹화
 - `isna`: 데이터가 NaN인지 아닌지 알려줌
 - `fillna()`: 결손 데이터를 다른 값으로 대체
 - **apply lambda**: `apply()` 로 각 값에 `lambda` 함수 적용



댓글을 작성하세요

댓글 작성

